

CSC-123 — Lab 01

Methods as “Reasoning Containers” (Types + Casting + Expression Evaluation)

Estimated Time: 60–90 minutes

Individual work

Submit exactly one file: 221lab01`USERNAME`.java

Purpose

Turn the reasoning into reusable, testable units (methods). You will practice:

- Method contracts (parameters/return types)
 - Where casts belong (inside methods, not “fixed later”)
 - Consistent formatted output
 - Avoiding duplicated logic
-

Requirements

Your program must define and use **these methods** (exact names and signatures):

```
public static double divAsDouble(int a, int b)
public static int divAsInt(int a, int b)
public static String classifyDivision(int a, int b)
public static void printDivisionReport(int a, int b)
public static void printDivisionReport(int a, int b)
```

Method behavior

1. `divAsDouble(int a, int b)`
 - Returns `(double)a / b`
 2. `divAsInt(int a, int b)`
 - Returns `a / b` (integer division)
 3. `classifyDivision(int a, int b)`
 - Returns "exact" if `a % b == 0`, otherwise "truncates"
 4. `printDivisionReport(int a, int b)`
 - Prints a 4-line report using `printf` with labels and 2 decimals:
 - `a / b`
 - `(double)a / b`
 - `a / (double)b`
 - `(double) (a / b)`
 - Also prints the classification line:
 - `classification = exact` **OR** `classification = truncates`
-

Input/Output spec (for autograding)

Program reads **two integers** from standard input: `a` then `b`.

Output must include these labels exactly (even whitespace your autograder is strict):

```
a/b = ...
(double)a/b = ...
a/(double)b = ...
(double) (a/b) = ...
classification = ...
```

Constraints

- Must use `printf` for the numeric lines.
 - Must not hard-code results.
 - Must call the methods (no duplicated inline logic in `main`).
-

Edge cases

- If `b == 0`: print
ERROR: division by zero
and exit (no stack trace, no crash).
-

Submission

Upload exactly one Java file named:

221proj02`USERNAME`.java

`USERNAME` is your lab login (first part of campus email).

Test cases (recommended)

Use 5 tests to catch the classic misconceptions:

1. **Integer truncation**
 - Input: `7 2`
 - Expect: integer division line differs from double division lines
2. **Exact division**
 - Input: `8 2`
 - Expect: integer and double match (2-decimal formatting)
3. **Negative division**
 - Input: `-7 2`
 - Expect: truncation toward zero behavior visible
4. **Reverse**
 - Input: `2 7`
 - Expect: integer division becomes 0
5. **Zero divisor**
 - Input: `5 0`
 - Expect: `ERROR: division by zero`

Rough draft of submission rubric (for canvas)

(will scale to points, to total possible)

1. **Compiles & runs (2 pts)**
 - 2: Compiles cleanly, runs without crashing
 - 0: Does not compile or crashes
2. **Correct numeric behavior (3 pts)**
 - 3: All four division variants correct for standard tests
 - 2: One incorrect due to cast placement
 - 1: Multiple incorrect but some correct
 - 0: Fundamentally wrong
3. **Formatted output via `printf` (2 pts)**
 - 2: Uses `printf`, correct specifiers, readable labeling
 - 1: Uses formatting inconsistently or wrong specifiers
 - 0: No formatted output / unreadable
4. **Predictions present before execution (Part B) (2 pts)**
 - 2: Prediction comments exist for each expression
 - 1: Some predictions missing
 - 0: No predictions
5. **Reasoning summary (1 pt)**
 - 1: Final comment explains wrong predictions and why
 - 0: Missing or content-free

Hard rule (submission compliance)

- **-2 pts** if file name does not match `221proj02USERNAME.java`
- **0** if more than one file is submitted